# 설계/프로젝트 진행 보고서

## 틈 (Teum)

제안 상품명 : 틈(Teum) : 공강 시간 교집합 찾기 서비스
발 주 자 : 명지대학교 컴퓨터공학과 김상귀 교수
보고기관 : 명지대학교 컴퓨터공학과
설 계 팀 : Finder
작성일자 : 2026.09.24.~2026.10.12

<제목 차례>


**1. 서론 ····· 3**
가. 목적 ····· 3
나. 설계 프로젝트 팀의 팀명, 팀 구성수, 팀원 성명과 배경 ····· 3
다. 설계 목표 ····· 3
라. 보고서 구성 ····· 4
**2. 설계/프로젝트의 현실적 제한조건 ····· 4**
**3. 설계 결정에 대한 진행 상황 ····· 5**
가. Entity Relationship Diagram ····· 5
나. Class Diagram ····· 9
다. 문제해결에 적용된 이론 ····· 18
라. SW를 이용한 설계/해석 ····· 18
**4. 설계 평가 ····· 20**
가. 핵심 알고리즘 성능 및 정확도 평가 ····· 20
나. 예외 상황 처리 및 차선책 도출 검증 ····· 20
다. 데이터 무결성 및 변환 평가 ····· 20
라. 활용성 평가 ····· 20
**5. 설계/프로젝트 계획 ····· 21**
가. 서비스 개요 ····· 21
나. 앞으로의 개발 계획 ····· 21
다. 예상 일정 ····· 21
**6. 참고자료 ····· 22**
가. 학술적 근거 및 참고 문헌 ····· 22
나. 애자일(Agile) MVP (최소 기능 제품) 개발 방법론 ····· 22



**요약문**

본 보고서는 대학생의 팀 프로젝트와 스터디 모임을 위한 공강 시간 자동 추천 웹 서비스 '틈(Tuem)'의 상세 설계와 개발 계획을 제시한다. 시간표를 비트 벡터로 변환하여 공통 공강 시간을 계산하고, 이동 여유 시간과 최소 회의 시간을 반영한 후보 추천, 차선안 제시, 투표 및 최종 일정 확정 기능을 설계하였다. 유스케이스 다이어그램, ERD, 클래스 다이어그램을 통해 데이터 구조와 객체 간 관계를 정의하고, 데이터 무결성과 시간표 변경에 따른 재검사 방안을 마련하였다. 향후 핵심 기능을 구현하고 정확도 및 성능을 검증하여 대학생의 일정 조율 부담을 줄이고자 한다.

## Abstract

This report presents the detailed design and development plan for 'Tuem,' an automatic free-period-time recommendation web service for college students' team projects and study groups. By converting the timetable into a bit vector, common gap time was calculated, and functions for candidate recommendation, alternative proposal, voting, and final schedule confirmation were designed, taking into account travel buffer time and minimum meeting duration. Through use case diagrams, ERDs, and class diagrams, the data structure and relationships between objects were defined, and measures for data integrity and re-verification in response to timetable changes were established. In the future, we aim to implement key features and verify accuracy and performance to reduce the burden of scheduling for university students.

# 1. 서론

## 가. 목적

본 프로젝트는 대학생 그룹을 대상으로 팀원 각자의 강의 시간표를 분석하여 조건을 만족하는 공통 참석 가능 시간대를 자동 추천하는 웹사이트 '틈 (Teum)'을 설계하는 데 목적을 둔다.

## 나. 설계 프로젝트 팀의 팀명, 팀 구성수, 팀원 성명과 배경

1) 팀명: Finder
2) 팀 구성원 수 : 5명
   가) 팀장 한규진:
      (1) 배경 소개: 자료구조 과목 이수. 프로젝트 전체 리딩 및 요구사항분석 경험 보유.
      (2) 업무 분담: 프로젝트 총괄(PM),요구사항 정의 및 제안서 작성, 전체 시스템 구조 설계 지원
   나) 팀원 송하성:
      (1) 배경 소개: 웹 프론트엔드/UI 구현 및 설문 데이터 분석 관련 수업 이수.
      (2) 업무 분담: 사용자 요구사항 분석(설문 분석),웹 UI/UX 인터페이스 설계 및 입력 환경 구현
   다) 팀원 이다인:
      (1) 배경 소개: 비트 연산 및 핵심 매칭 알고리즘 최적화, 자료구조(힙/비트마스크)구현 관심 보유.
      (2) 업무 분담: 핵심 시간표 매칭 알고리즘 설계 및 점수 함수 연산 모델링
   라) 팀원 전기헌:
      (1) 배경 소개: 웹 기반 인터페이스 구축 및 텍스트/CSV데이터 파싱 및 직렬화 구현 경험 보유
      (2) 업무 분담: 사용자 시간표 데이터 입·출력 처리부(CSV/비트 변환기) 구현 및 성능 검증
   마) 팀원 현고은:
      (1) 배경 소개: 소프트웨어 공학 문서화 및 웹 플랫폼 테스트/품질 관리 관심 보유.
      (2) 업무 분담: 선행 서비스 분석 및 비교 조사, 시스템 통합 테스트케이스 작성 및 보고서 작성

## 다. 설계 목표

대학생들의 팀 프로젝트 및 스터디 모임은 매 학기 필수적으로 발생하며, 이에 따른 일정 조율 수요에 주목하였다. 최근 학생들의 전반적인 디지털 툴 활용도는 크게 높아졌다. 단체 채팅방에 시간표를 캡처해 눈으로 대조하는 수작업 방식이나 “When2meet”과 같은 단순 공유 수준의 일정 관리 도구는 이미 확정된 시간표를 일일이 다시 옮겨 적어야 하는 이중 입력의 번거로움을 유발한다. 또한 잦은 정보 누락과 일정 충돌, 그리고 의사소통 지연이라는 고질적인 문제를 지니고 있다.

이에 따라 '틈(Teum)'은 이러한 사용자 요구의 변화와 틈새를 포착하여 기존 시간표 데이터의 재활용성과 자체 알고리즘 추천을 결합한 새로운 대안을 제시한다. 이를 통한 일정 조율에 소요되는 시간 단축을 1차적 목표로 하며 향후 대규모 시장 확장을 기대할 수 있다.

### 라. 보고서 구성

본 보고서는 다음과 같은 흐름으로 구성된다. 제1장 서론에서는 프로젝트의 전반적인 개요와 목적, 설계 지원 주체 및 주요 사용자를 비롯하여 프로젝트의 필요성과 기대효과 등을 포괄적으로 밝힌다. 제2장에서는 안정적인 시스템 구현을 위해 극복해야 할 현실적 제한조건을 관리적 제한과 시스템적 제한(데이터 규격, 입력 환경, 인프라, 성능 제한 등)으로 명확히 구분하여 명시한다. 이어 제3장에서는 ERD와 클래스 다이어그램(Class Diagram)을 중심으로 한 데이터 모델링 및 아키텍처 구조를 통해 설계 결정에 대한 구체적인 진행 상황을 기술한다. 마지막으로 제4장 설계 평가와 제5장 설계/프로젝트 계획을 통해 향후 검증 기준과 남은 개발 로드맵을 제시하는 순서로 마무리된다.

## 2. 설계/프로젝트의 현실적 제한조건

안정적인 웹사이트 형태의 MVP(최소 기능 제품) 구현과 시스템 효율을 위해 다음과 같은 현실적 제한조건을 정량적으로 설정한다.

#### 1) 관리적 제한

가) 팀원 및 개발 기간 제한: 5명의 재학생 팀원이 한 학기(약 12주) 내에 MVP 모델로 필수 기능 개발을 완료해야 한다.

나) 개발 공정 제한: 기한 내 프로젝트 완수를 위해 전체 일정을 1-4주차(설계 및 구조화), 5-7주차(핵심 기능 구현), 8-9주차(통합 및 시각화), 10-12주차(성능 검증 및 시연)로 세분화하여 각 단계별 목표를 엄격히 준수한다.

#### 2) 시스템적 제한

가) 인프라 및 구조 제한 : 기존 제안서의 '데이터베이스 미구축' 조건에서 선회하여, 서비스 확장성과 데이터 보존을 위해 공식적인 데이터베이스 사용한다.

나) 입력 환경 및 예외 처리 제한: 별도의 프로그램 설치가 없는 웹 환경을 기반으로 한다. 에브리타임 시간표 캡처본을 바탕으로 일정을 추출하는 이미지 인식 방식과 강의 외 개인 일정(과외, 알바 등)을 직접 추가할 수 있는 수기 입력을 병행 지원한다.또한, 시간표 미입력자가 '전원 참석 가능'으로 잘못 연산되지 않도록 엄격한 예외 처리를 설정한다.

다) 데이터 규격 및 알고리즘 제한: 모든 일정 데이터는 고정 단위 슬롯으로 분할한 뒤, 참석 가능(1)과 불가능(0) 상태의 비트열로 규격화해야 한다. 공통 시간 탐색 시 단순 교집합 연산뿐만 아니라 이동 여유 시간과 최소 회의 시간 알고리즘을 필수적으로 거치도록 제한한다.

라) 성능 제한: 대규모 사용자 확장을 고려하여, 모임 인원이 증가하더라도 핵심 연산 시간 및 UI 응답 시간이 지연되지 않도록 알고리즘 최적화를 통해 일관되고 안정적인 연산 성능을 보장해야 한다.

# 3. 설계 결정에 대한 진행 상황

## 가. Entity Relationship Diagram

데이터베이스의 구조와 데이터 간의 관계를 시각적으로 표현하였다.




<그림1 - ERD>

### 1) 사용자 시간표 영역

#### 가) USER

소셜 로그인으로 가입한 사용자 정보와 시간표 등록 상태를 관리한다.

| | 핵심 필드 | 설계 포인트 | 관련 유스케이프 |
|---|---|---|---|
| USER | provider·provider_user_id(소셜 계정 식별, 복합 유니크), name·major(프로필 필수 정보), profile_completed(모임 참여 가능 여부 판단), timetable_registered_at(NULL이면 시간표 미등록), timetable_version(시간표 수정 시 +1), deleted_at(탈퇴 처리) | 소셜 계정은 (provider, provider_user_id) 조합으로 식별하므로, 서로 다른 플랫폼에서 같은 ID가 나와도 충돌하지 않는다. | 소셜 로그인, 로그아웃, 프로필 조회·수정, 시간표 가져오기·입력 |

<표1 - ERD 사용자 영역_User>

### 나) 시간표 일정

사용자의 주간 시간표를 요일 · 시간 단위로 저장한다. 한 행이 수업 하나로 제한한다.

| | 핵심 필드 | 설계 포인트 | 관련 유스케이프 |
|---|---|---|---|
| TimeTable Slot | day_of_week(요일, 0~6), start_time·end_time(수업 시간), title(과목명) | - 시간표는 등록했지만 일정이 0건인 사용자는 "공강이 많은 사람"으로 처리되어, 미등록 사용자와 구분한다. - 시작 시각이 종료 시각보다 늦은 데이터는 CHECK 제약으로 막는다. | 시간표 가져오기, 직접 입력, 수정 |

<표2 -ERD 사용자 시간표 영역_시간표 일정>

## 2) 모임 · 참여 영역

### 가) 모임

모임의 기본정보,공강 탐색조건, 초대 및 공유정보를 함께 관리한다.

| | 핵심 필드 | 설계 포인트 | 관련 유스케이프 |
|---|---|---|---|
| Meeting | name·description(모임 정보), period_start~end(조회 기간), search_start~end_time(탐색 시간 범위), travel_buffer_min(이동 여유 시간), min_duration_min(최소 회의 시간), needs_recalc(재계산 필요 상태), invite_code·invite_expires_at(초대 정보), share_token(결과 공유 링크), deleted_at(모임 삭제) | - 구성원이 시간표를 수정하면 같은 트랜잭션 안에서 needs_recalc만 true로 바꾸고, 실제 재계산은 관리자가 실행한다. -일정 확정 여부는 별도 상태 컬럼 없이 CONFIRMED_SCHEDULE 존재 여부로 판단한다. | 모임 생성·수정·삭제, 초대 링크/코드 생성, 결과 링크 공유 |

<표3 - ERD 모임 참여 영역_Meeting>

### 나) 모임 구성원

사용자와 모임을 연결하고 모임 안에서의 역할을 구분한다.

| | 핵심 필드 | 설계 포인트 | 관련 유스케이프 |
|---|---|---|---|
| Meeting Member | role(ADMIN/MEMBER), status(ACTIVE/LEFT) | -(meeting_id, user_id) 복합 유니크로 중복 가입을 막는다. -관리자도 구성원 테이블의 한 행이라서 구성원 공통 기능을 그대로 사용할 수 있다. -관리자 위임은 기존 관리자와 새 관리자의 role 변경을 한 트랜잭션으로 처리한다. -관리자가 1명뿐이면 위임하거나 모임을 삭제하기 전에는 탈퇴할 수 없다. | 모임 참여, 모임 탈퇴, 관리자 위임 |

<표4 - ERD 모임 참여 영역_모임구성원>

### 3) 매칭 투표 확정 영역

#### 가) 매칭 계산 이력

공강 시간 자동 매칭을 실행할 때마다 한 행씩 쌓이는 계산 기록이다.

| | 핵심 필드 | 설계 포인트 | 관련 유스케이프 |
|---|---|---|---|
| Matching Run | trigger_type(참여/관리자 재계산/확정 전 재검사), cond_snapshot(계산 당시 탐색 조건, JSON), total_member_count·timetable_input_count(시각화 안내용 인원), input_hash(계산 당시 시간표 상태), calculated_at(마지막 계산 시각) | - 계산 결과를 덮어쓰지 않고 이력으로 남기기 때문에, 조건이나 구성원이 바뀌어도 이전 결과와 섞이지 않는다. -최신 결과는 (meeting_id, calculated_at) 인덱스로 빠르게 조회한다. | 시간표 자동 매칭, 공강 시간 다시 계산, 최신 시간표 기준 재검사 |

<표5 - ERD 매칭투표 확정 영역_매칭계산 이력>

#### 나) 후보 묶음

한 번의 계산으로 만들어진 후보들을 투표 라운드 단위로 묶는다.

| | 핵심 필드 | 설계 포인트 | 관련 유스케이프 |
|---|---|---|---|
| Candidate Set | set_type(일반 추천/차선책), reason(차선책 진입 사유), result_status(후보 있음/없음), vote_status(투표 중/마감/폐기) | -계산하면 기존 묶음은 "폐기"로 바꾸고 새 묶음이 생긴다. -이전 투표는 기존 묶음에 보관한다. | 추천 시간대 제시, 차선책 제안 |

<표6 - ERD 매칭투표 확정 영역_후보 묶음>

#### 다) 후보 시간대

추천되는 회의 시간 하나하나를 저장한다. 묶음당 최대 3개로 제한한다.

| | 핵심 필드 | 설계 포인트 | 관련 유스케이프 |
|---|---|---|---|
| Candidate | rank_no(순위), start_at·end_at(시간대), available_count(참석 가능 인원) | -(set_id, rank_no) 복합 유니크로 같은 묶음 안에서 순위가 겹치지 않는다. -불참 인원은 MATCHING_RUN의 전체 인원에서 참석 가능 인원을 빼서 계산한다. | 추천 시간대 제시, 차선책 제안, 매칭 결과 조회 |

<표7 - ERD 매칭투표 확정 영역_후보 시간대>

#### 라) 투표지

구성원 한 명이 후보 묶음 하나에 제출한 투표 1건으로 제한한다

| | 핵심 필드 | 설계 포인트 | 관련 유스케이프 |
|---|---|---|---|
| Vote Ballot | set_id(어느 묶음에 대한 투표인지), member_id(투표자), no_available(선택 가능한 후보 없음) | -(set_id, member_id) 복합 유니크로 한 묶음에 구성원 한 명당 투표지는 하나로 제한한다. | 추천 시간대 선호 투표, 투표 결과 조회 |

<표8 - ERD 매칭투표 확정 영역_투표지>

**마) 투표 선택 항목**

투표지에서 선택한 후보를 저장한다.

| | 핵심 필드 | 설계 포인트 | 관련 유스케이프 |
|---|---|---|---|
| Vote Selection | ballot_id + candidate_id(복합 PK), set_id(정합성 검증용) | - 투표지와 후보를 각각 set_id와 함께 복합 FK로 참조하므로, 다른 묶음의 후보를 선택하는 데이터는 DB 단계에서 차단된다.<br>-"후보 없음"을 선택한 투표지에는 선택 항목이 없다. | 추천 시간대 선호 투표 |

<표9 - ERD 매칭투표 확정 영역_ 투표 선택 항목>

**바) 확정 일정**

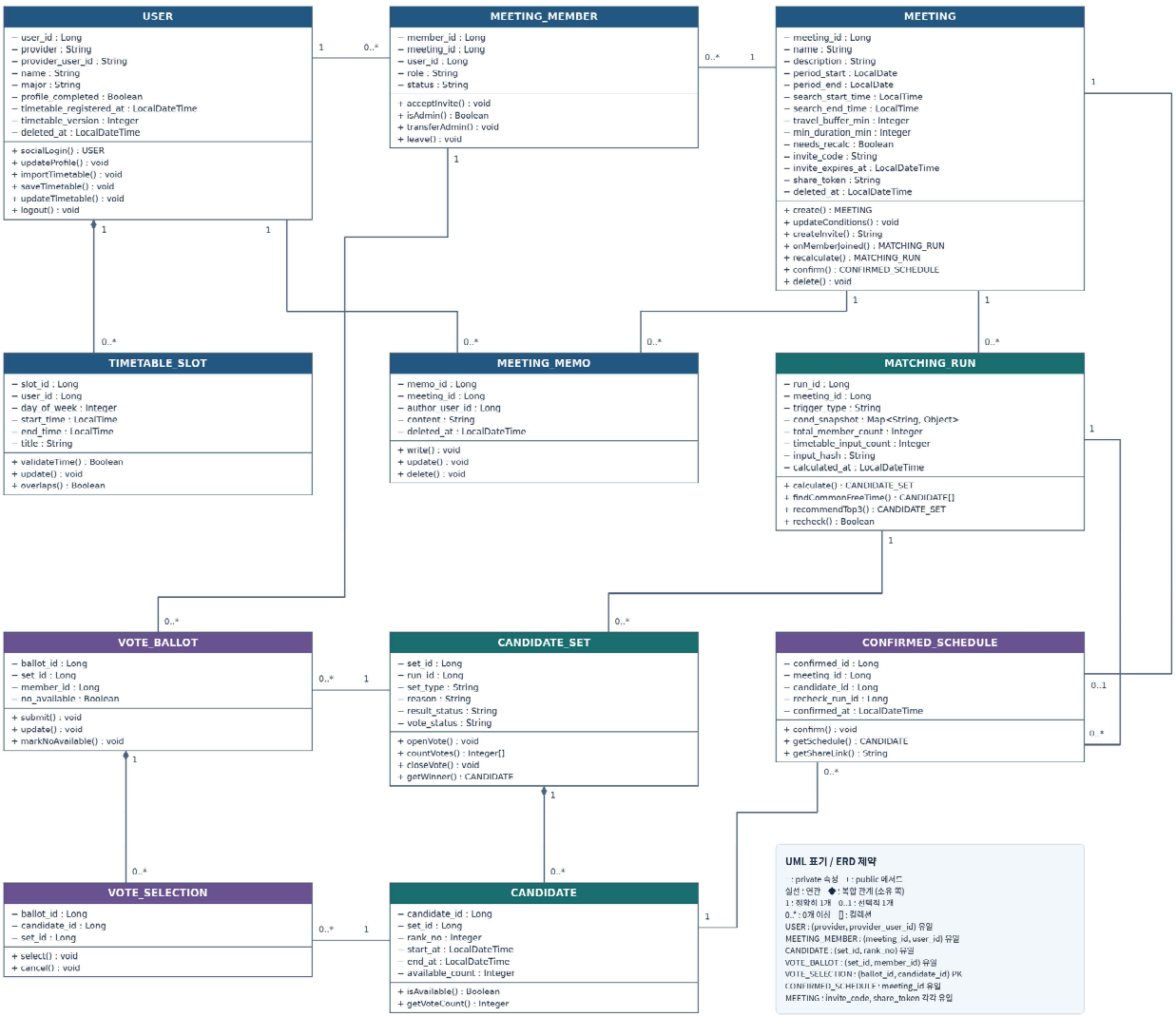
관리자가 최종 확정한 모임 일정이다. 모임당 1개로 제한한다.

| | 핵심 필드 | 설계 포인트 | 관련 유스케이프 |
|---|---|---|---|
| Confirmed Schedule | candidate_id(확정된 후보), recheck_run_id(확정 직전 재검사 기록), confirmed_at(확정 시각) | - 확정 직전에 최신 시간표로 재검사한 결과를 함께 남겨, 확정 시점의 신뢰성을 보장한다.<br>- meeting_id는 조회 편의와 "모임당 1개" 제약을 위해 의도적으로 함께 저장한다. | 최종 일정 확정, 확정 일정 조회 |

<표10 - ERD 매칭투표 확정 영역_ 확정 일정>

## 나. Class Diagram

draw.io라는 프로그램을 이용해서 클래스 간의 관계를 정의하여 구체적으로 프로그래밍을 할 때 필요한 정보들을 정리해 클래스 다이어그램을 만들었다.




<그림2 - Class Diagram>

### 1) 클래스 사이의 관계

클래스 사이의 선은 객체들이 어떤 관계를 맺는지 나타내며, 선 양 끝의 숫자는 연결될 수 있는 객체 수인 '다중성'을 의미한다. 검은 마름모는 전체에 해당하는 클래스 쪽에 표시한다. 이번 다이어그램의 연관선에는 방향 화살표가 없으므로, 선 자체로 참조 방향을 제한하지 않는다. 또한 이 선들은 실행 순서를 나타내는 흐름도가 아니다.

| 표기 | 의미 |
|---|---|
| 실선 | 두 클래스가 연결되어 정보를 참조하는 연관 관계 |
| 검은 마름모가 있는 실선 (◆) | 전체 객체가 부분 객체를 소유하는 복합관계 |
| 1 | 정확히 한개 |
| 0..1 | 없거나 한개 |
| 0..* | 없거나 여러개 |

<표11 - Class Diagram_관계>

| 번호 | 클래스 관계 | 다중성 | 설명 |
|---|---|---|---|
| ① | USER - MEETING_MEMBER | 1 - 0..* | - 사용자 한 명은 여러 모임에 참여할 수 있으므로 여러 개의 참여 정보를 가질 수 있다.<br>- 각 참여 정보는 정확히 한 명의 사용자에게 속한다. 아직 모임에 참여하지 않은 사용자는 참여 정보가 없을 수 있다. |
| ② | MEETING - MEETING_MEMBER | 1 - 0..* | - 하나의 모임은 여러 구성원의 참여 정보를 가질 수 있다. 각 참여 정보는 정확히 하나의 모임에 속한다. USER와 MEETING 사이의 다대다 관계를 MEETING_MEMBER가 연결한다. |
| ③ | USER ◆- TIMETABLE_SLOT | 1 - 0..* | - 사용자는 여러 시간표 항목을 소유한다. 각 항목은 한 사용자에게만 속하며, 해당 사용자의 시간표를 구성하는 부분이므로 복합 관계로 표현한다.<br>- 시간표를 입력하지 않은 사용자는 항목이 없을 수 있다. |
| ④ | USER - MEETING_MEMO | 1 - 0..* | - 사용자 한 명은 여러 메모를 작성할 수 있다.<br>- 각 메모에는 정확히 한 명의 작성자가 연결된다. |
| ⑤ | MEETING - MEETING_MEMO | 1 - 0..* | - 하나의 모임에는 여러 메모가 등록될 수 있으며, 각 메모는 정확히 하나의 모임에 속한다. |
| ⑥ | MEETING - MATCHING_RUN | 1 - 0..* | - 모임은 구성원 참여나 관리자의 재계산 요청에 따라 공강 계산을 여러 번 실행할 수 있다.<br>- 각 계산 이력은 정확히 하나의 모임에 연결된다. |
| ⑦ | MEETING_MEMBER - VOTE_BALLOT | 1 - 0..* | - 구성원은 서로 다른 후보 집합에 대해 여러 투표 응답을 가질 수 있다.<br>- 각 투표 응답의 투표자는 정확히 한 명이다.<br>- 단, 동일 후보 집합에서는 구성원당 하나의 투표 응답만 허용한다. |
| ⑧ | MATCHING_RUN - CANDIDATE_SET | 1 - 0..* | - 하나의 계산 실행은 추천안이나 차선안 등의 후보 집합을 생성한다.<br>- 각 후보 집합은 정확히 하나의 계산 실행에서 생성된다. |

| 번호 | 클래스 관계 | 다중성 | 설명 |
|---|---|---|---|
| ⑨ | CANDIDATE_SET - VOTE_BALLOT | 1 - 0..* | - 하나의 후보 집합에는 여러 구성원의 투표 응답이 연결된다.<br>- 각 투표 응답은 정확히 하나의 후보 집합을 대상으로 한다. |
| ⑩ | CANDIDATE_SET ◆- CANDIDATE | 1 - 0..* | - 후보 집합은 개별 일정 후보들을 소유한다.<br>- 각 후보는 정확히 하나의 후보 집합에 속하는 구성 요소이므로 복합 관계로 표현한다.<br>- 추천 후보를 최대 3개로 제한하는 규칙은 별도의 업무 제약으로 적용한다. |
| ⑪ | VOTE_BALLOT ◆- VOTE_SELECTION | 1 - 0..* | - 하나의 투표 응답은 실제 선택한 후보들을 선택 항목으로 소유한다.<br>- 선택 항목은 특정 투표 응답에 종속되므로 복합 관계로 표현한다.<br>- 참석 가능한 후보가 없다고 응답하면 선택 항목은 0개가 된다. |
| ⑫ | CANDIDATE - VOTE_SELECTION | 1 - 0..* | - 하나의 후보는 여러 구성원에게 선택될 수 있다.<br>- 각 선택 항목은 정확히 하나의 후보를 가리키며, 이를 이용해 후보별 득표수를 집계한다. |
| ⑬ | CANDIDATE - CONFIRMED_SCHEDULE | 1 - 0..* | - 각 확정 일정은 정확히 하나의 후보를 참조한다.<br>- 현재 그림은 ERD의 연결을 따라 후보 쪽에서 여러 확정 기록을 허용하는 형태로 표시되어 있다.<br>- 다만 후보가 생성된 모임에서만 확정할 수 있고 모임당 확정 일정이 하나라는 업무 규칙까지 엄격히 반영한다면, 확정 일정 쪽 다중성은 0..1로 좁힐 수 있다. |
| ⑭ | MEETING - CONFIRMED_SCHEDULE | 1 - 0..1 | - 하나의 모임은 최대 하나의 확정 일정을 가진다.<br>- 일정 확정 전에는 확정 일정이 없으며, 각 확정 일정은 정확히 하나의 모임에 속한다. |
| ⑮ | MATCHING_RUN - CONFIRMED_SCHEDULE | 1 - 0..* | - 각 확정 일정은 최종 확정 전에 수행한 하나의 재검사 이력을 참조한다.<br>- 이를 통해 어떤 계산 결과를 근거로 일정이 확정되었는지 확인한다.<br>- 그림에서는 하나의 계산 이력을 여러 확정 기록이 참조할 수 있는 구조로 표현한다. |

<표12 - Class Diagram_설명>

## 2) 클래스별 담당 역할, 속성 및 기능

### 가) User

사용자의 소셜 계정, 프로필 및 시간표 등록 상태를 관리한다. 개인 시간표와 모임 참여 정보를 연결하는 기준이 된다.

| 속성 | Java 자료형 | 설명 |
|---|---|---|
| user_id | Long | 사용자 고유 번호 |
| provider | String | 소셜 로그인 제공자 |
| provider_user_id | String | 제공자가 부여한 사용자 식별값 |
| name | String | 사용자 이름 |
| major | String | 전공 |
| profile_completed | Boolean | 프로필 작성 완료 여부 |
| timetable_registered_at | LocalDateTime | 시간표 등록시각 |
| timetable_version | Integer | 시간표 변경 전 |
| deleted_at | LocalDateTime | 계정 삭제 처리 시각 |

<표13 - Class Diagram_속성>

| 기능 | 설명 |
|---|---|
| socialLogin() : USER | 소셜 인증을 처리하고 해당 사용자 객체를 반환한다. |
| updateProfile() : void | 이름과 전공 등 프로필을 수정한다. |
| importTimetable() : void | 업로드한 시간표 이미지를 OCR 또는 AI로 분석하여 일정 정보를 추출한다. |
| saveTimetable() : void | 추출 결과 또는 직접 입력한 시간표를 저장한다. |
| updateTimetable() : void | 시간표와 버전을 갱신하고 소속 모임에 재계산이 필요함을 표시한다. |
| logout() : void | 로그아웃을 처리한다. |

<표14 - Class Diagram_기능 설명>

### 나) TimeTable_Slot - 시간표 항목

개인 시간표를 구성하는 개별 수업이나 일정을 관리한다. 공강 계산 시 참석할 수 없는 시간 구간을 판단하는 자료가 된다.

| 속성 | Java 자료형 | 설명 |
|---|---|---|
| slot_id | Long | 시간표 항목 고유 번호 |
| user_id | Long | 소유한 사용자의 고유 번호 |
| day_of_week | Integer | 일정이 있는 요일 |
| start_time | LocalTime | 시작시각 |
| end_time | LocalTime | 종료시각 |
| title | String | 수업명 또는 일정명 |

<표15 - Class Diagram_속성>

| 기능 | 설명 |
|---|---|
| validateTime() : Boolean | 시작·종료 시각의 유효성을 검사한다. |
| update() : void | 시간 정보를 수정한다. |
| overlaps() : Boolean | 비교 대상 시간 구간과 일정이 겹치는지 판단한다. |

<표16 - Class Diagram_기능 설명>

### 다) Meeting - 모임

모임의 기본 정보와 공강 검색 조건을 관리한다. 초대, 자동 계산, 관리자 수동 재계산 및 최종 일정 확정 과정을 조정한다.

| 속성 | Java 자료형 | 설명 |
|---|---|---|
| meeting_id | Long | 모임 고유 번호 |
| name | String | 모임 이름 |
| description | String | 모임 설명 |
| period_start | LocalDate | 검색 기간 시작일 |
| period_end | LocalDate | 검색 기간 종료일 |
| search_start_time | LocalTime | 하루 검색 범위의 시작 시각 |
| search_end_time | LocalTime | 하루 검색 범위의 종료 시각 |
| travel_buffer_min | Integer | 이동 여유 시간(분) |
| min_duration_min | Integer | 모임 최소 지속 시간(분) |
| needs_recalc | Boolean | 재계산 필요 여부 |
| invite_code | String | 고유 초대 코드 |
| invite_expires_at | LocalDateTime | 초대 코드 만료 시각 |
| share_token | String | 결과 공유용 고유 토큰 |
| deleted_at | LocalDateTime | 모임 삭제 처리 시각 |

<표17 - Class Diagram_속성>

| 기능 | 설명 |
|---|---|
| create() : MEETING | 모임을 생성하고 모임 객체를 반환한다. |
| updateConditions() : void | 공강 검색 조건을 수정한다. |
| createInvite() : String | 초대 코드를 생성하여 반환한다. |
| onMemberJoined() : MATCHING_RUN | 구성원 참여에 따른 자동 계산을 실행하고 계산 이력을 반환한다. |
| recalculate() : MATCHING_RUN | 관리자 권한 확인 후 최신 시간표로 공강을 재계산한다. |
| confirm() : CONFIRMED_SCHEDULE | 선택 후보의 확정 처리를 수행하고 확정 일정 객체를 반환한다. |
| delete() : void | 모임 삭제를 처리한다. |

<표18 - Class Diagram_기능 설명>

### 라) Meeting_Member - 모임 구성원

사용자와 모임의 참여 관계를 관리하며, 모임별 역할과 참여 상태를 구분한다.

| 속성 | Java 자료형 | 설명 |
|---|---|---|
| member_id | Long | 참여 관계 고유 번호 |
| meeting_id | Long | 참여한 모임의 고유 번호 |
| user_id | Long | 참여한 사용자의 고유 번호 |
| role | String | 관리자 ADMIN 또는 일반구성 Member |
| status | String | 참여중 Active 또는 타퇴 Left |

<표19 - Class Diagram_속성>

| 기능 | 설명 |
|---|---|
| acceptInvite() : void | 초대 정보와 시간표 등록 상태를 확인하고 모임 참여를 처리한다. |
| isAdmin() : Boolean | 관리자 여부를 확인한다. |
| transferAdmin() : void | 다른 구성원에게 관리자 권한을 위임한다. |
| leave() : void | 모임 탈퇴를 처리한다. |

<표20 - Class Diagram_기능 설명>

### 마) Meeting Memo - 모임 메모

구성원들이 공유하는 모임 안내 사항과 의견을 기록한다.

| 속성 | Java 자료형 | 설명 |
|---|---|---|
| memo_id | Long | 메모 고유 번호 |
| meeting_id | Long | 메모가 속한 모임의 고유 번호 |
| author_user_id | Long | 작성자의 사용자 고유 번호 |
| content | String | 메모 내용 |
| deleted_at | LocalDateTime | 메모 삭제 처리 시각 |

<표21 - Class Diagram_속성>

| 기능 | 설명 |
|---|---|
| write() : void | 메모를 등록한다. |
| update() : void | 메모 내용을 수정한다 |
| delete() : void | 메모 삭제를 처리한다. |

<표22 - Class Diagram_기능 설명>

### 바) Matching Run - 공강 계산 실행

시간표와 검색 조건을 이용해 공강을 계산하고, 계산 당시의 입력 상태와 실행 이력을 기록한다.

| 속성 | Java 자료형 | 설명 |
|---|---|---|
| run_id | Long | 계산 실행 고유 번호 |
| meeting_id | Long | 계산 대상 모임의 고유 번호 |
| trigger_type | String | 자동 계산, 수동 재 계산 등 실행 계기 |
| cond_snapshot | Map<String, Object> | 계산 당시 검색 조건 |
| total_member_count | Integer | 계산 당시 전체 구성원 수 |
| timetable_input_count | Integer | 시간표를 입력한 구성원 수 |
| input_hash | String | 입력 정보 변경 여부 비교용 해시값 |
| calculated_at | LocalDateTime | 계산 실행 시각 |

<표23 - Class Diagram_속성>

| 기능 | 설명 |
|---|---|
| calculate() : CANDIDATE_SET | 공강 계산을 수행하고 후보 집합을 반환한다. |
| findCommonFreeTime() : CANDIDATE[] | 공통 공강 시간대를 찾아 후보 배열을 반환한다. |
| recommendTop3() : CANDIDATE_SET | 추천 후보를 최대 3개 선정하여 후보 집합으로 반환한다. |
| recheck() : Boolean | 선택 후보가 최신 시간표에서도 유효한지 재검사한다. |

<표24 - Class Diagram_기능 설명>

### 사) Candidate_Set - 후보 집합

계산 결과로 생성된 후보들을 묶어 관리하고, 해당 집합에 대한 투표 진행과 집계를 담당한다.

| 속성 | Java 자료형 | 설명 |
|---|---|---|
| set_id | Long | 후보 집합 고유 번호 |
| run_id | Long | 후보 집합을 생성한 계산 실행 번호 |
| set_type | String | 추천안 또는 차선안 |
| reason | String | 후보집합을 제시한 이유 |
| result_status | String | 계산 결과 상태 |
| vote_status | String | 튜표 진행 상태 |

<표25 - Class Diagram_기능 설명>

| 기능 | 설명 |
|---|---|
| openVote() : void | 시작 조건을 확인하고 투표를 시작한다. |
| countVotes() : Integer[] | 후보별 득표수를 집계하여 배열로 반환한다. |
| closeVote() : void | 투표를 종료한다. |
| getWinner(): CANDIDATE | 투표 결과에 따라 선택된 후보를 반환한다. |

<표26 - Class Diagram_속성>

### 아) Candidate - 개별 시간대 후보

추천 또는 차선안에 포함되는 개별 시간대를 관리한다. 구성원의 선호 투표와 최종 일정 선택의 대상이 된다.

| 속성 | Java 자료형 | 설명 |
|---|---|---|
| candidate_id | Long | 후보 고유 번호 |
| set_id | Long | 소속 후보 집합 번호 |
| rank_no | Integer | 후보 집합 내 추천 순위 |
| start_at | LocalDateTime | 후보 일정 시작 일시 |
| end_at | LocalDateTime | 후보 일정 종료 일시 |
| available_count | Integer | 참석 가능한 구성원 수 |

<표27 - Class Diagram_기능 설명>

| 기능 | 설명 |
|---|---|
| isAvailable() : Boolean | 확인 대상 구성원이 해당 후보 시간대에 참석 가능한지 판단한다. |
| getVoteCount() : Integer | 해당 후보의 득표수를 반환한다. |

<표28 - Class Diagram_속성>

### 자) Vote_Ballot - 투표 응답

특정 후보 집합에 대한 구성원 한 명의 투표 응답을 관리한다. 참석 가능한 후보가 없다는 응답도 기록한다.

| 속성 | Java 자료형 | 설명 |
|---|---|---|
| ballot_id | Long | 투표 응답 고유 번호 |
| set_id | Long | 투표 대상 후보 집합 번호 |
| member_id | Long | 투표한 구성원의 고유 번호 |
| no_available | Boolean | 참석 가능한 후보가 없는지 여부 |

<표29 - Class Diagram_기능 설명>

| 기능 | 설명 |
|---|---|
| submit() : void | 선택한 후보를 투표 응답으로 제출한다. |
| update() : void | 기존 투표 내용을 변경한다. |
| markNoAvailable() : void | 참석 가능한 후보가 없음을 기록하고 선택 항목을 비운다. |

<표30 - Class Diagram_속성>

### 차) Vote_Selection - 투표 선택 항목

투표 응답에서 실제로 선택한 후보를 기록한다. 투표 응답과 후보를 연결하여 득표 집계에 활용한다.

| 속성 | Java 자료형 | 설명 |
|---|---|---|
| ballot_id | Long | 소속 투표 응답 번호 |
| set_id | Long | 투표 응답과 후보가 속한 후보 집합 번호 |
| candidate_id | Long | 선택한 후보 번호 |

<표31 - Class Diagram_기능 설명>

| 기능 | 설명 |
|---|---|
| select() : void | 후보 선택을 추가하고 투표 응답과 후보가 동일 집합에 속하는지 확인한다. |
| cancel() : void | 해당 후보 선택을 취소한다. |

<표32 - Class Diagram_속성>

### 카) Confirmed_Schedule - 확정 일정

투표 결과와 최신 시간표 재검사를 거쳐 확정된 일정을 관리한다. 확정 후보와 검증 이력을 연결하고 일정 조회 및 공유를 지원한다.

| 속성 | Java 자료형 | 설명 |
|---|---|---|
| confirmed_id | Long | 확정 일정 고유 번호 |
| meeting_id | Long | 일정이 확정된 모임 번호 |
| candidate_id | Long | 최종 선택된 후보 번호 |
| recheck_run_id | Long | 확정 전 재검사 실행 번호 |
| confirmed_at | LocalDateTime | 일정 확정 시각 |

<표33 - Class Diagram_기능 설명>

| 기능 | 설명 |
|---|---|
| confirm() : void | 재검사 결과를 확인하고 최종 일정을 저장한다. |
| getSchedule() : CANDIDATE | 확정된 일정에 연결된 후보 객체를 반환한다. |
| getShareLink() : String | 확정 결과 공유 링크를 반환한다. |

<표34 - Class Diagram_속성>

## 3) 설계 참고 사항

가) LocalDate, LocalTime, LocalDateTime은 각각 날짜, 시각, 날짜와 시각을 표현한다.

나) Map<String, Object>는 검색 조건을 이름과 값의 쌍으로 관리하기 위한 설계 선택이다.

다) 메서드의 void는 반환값이 없음을, []는 배열을 의미한다.

라) 현재 모델에는 초대 대상과 미수락 상태를 기록하는 구조가 없으므로, '초대받은 팀원 전원이 수락했는지'를 판정하려면 해당 정보를 관리하는 구조를 보완해야 한다. 또한 메서드의 매개변수 생략은 다이어그램을 간결하게 표현하기 위한 것이며, 실제 구현에서 입력 정보가 불필요하다는 의미는 아니다.

## 다. 문제해결에 적용된 이론

### 1) 비트 벡터(Bit Vector)와 워드 단위 비트 연산을 활용한 공강 계산 최적화

#### 가) 적용 기술 및 데이터 모델링

대학생의 주간 시간표를 120개의 시간 슬롯으로 구분하고, 각 슬롯의 참석 가능 여부를 1(가능)과 0(불가능)으로 표현하는 비트 벡터(Bit Vector)로 모델링한다. 각 참여자의 시간표는 비트 벡터로 저장하며, 64비트 워드를 기준으로 하면 약 120개의 슬롯을 2~3개의 워드에 표현할 수 있다.

#### 나) 공통 참석 가능 시간 계산

모든 참여자의 공통 참석 가능 시간은 각 시간표의 비트 벡터에 비트 AND 연산을 반복 적용하여 계산한다. 각 비트 위치에서 모든 참여자의 값이 1인 경우에만 결과가 1이 되므로, 최종 비트 벡터에서 1로 표시된 위치가 전체 참여자의 공통 참석 가능 시간에 해당한다.

비트 연산은 하나의 워드에 포함된 여러 비트를 동시에 처리할 수 있다. 따라서 슬롯을 하나씩 비교하는 방식과 달리, 64비트 워드 기준으로 최대 64개의 슬롯을 한 번의 AND 연산으로 처리할 수 있다. 전체 시간 슬롯 수를 (S), 참여자 수를 (N), 워드 크기를 (w)라고 하면, 공통 시간표의 교집합 계산에 필요한 워드 연산 횟수는 O(N × ⌈S/w⌉)이며, ⌈S/w⌉는 필요한 워드 수를 뜻한다.

본 시스템에서 (S)는 120개로 고정된 범위이므로, 64비트 워드 기준 공통 시간 계산은 참여자 수(N)에 대해 선형적인 시간 복잡도 (O(N))을 갖는다. 이 기법의 핵심은 점근적 시간 복잡도를 단순히 낮추는 것보다는, 여러 시간 슬롯을 워드 단위로 처리해 반복적인 슬롯별 비교 연산을 줄이는 데 있다.

## 라. SW를 이용한 설계/해석

12주라는 제한된 시간 내에 완성도 높은 MVP(최소 기능 제품)를 만들기 위해 적용한 소프트웨어 공학적 설계 방법론이다.

### 1) 애자일(Agile) MVP (최소 기능 제품) 개발 방법론

애자일 방법론은 전통적인 폭포수 방식과 달리, 변화에 유연하게 대응하고 사용자의 의견을 지속적으로 반영하며 개발을 진행하는 방식을 지향한다. 틈(Teum) 프로젝트는 다음과 같은 이유로 애자일 방법론을 채택하였다

*폭포수 방식 : 프로젝트의 시작부터 끝까지 각 단계를 순차적이고 선형적으로 진행하는 전통적인 소프트웨어 개발·프로젝트 관리 방식

#### 가) 개인과 상호작용

5명의 팀원이 한 학기(12주)라는 짧은 기간 동안 협력해야 하는 환경에서는 복잡한 개발 프로세스나 도구 사용법에 얽매이기보단 팀원 간의 원활한 소통과 유기적인 협업을 통해 알고리즘 설계와 웹사이트 화면 구현을 신속하게 조율한다.

#### 나) 우선순위 반복 개발

방대한 설계 문서 작성에 지나치게 시간을 쏟기보다는, 핵심 기능인 비트 연산 기반 공강 시간 자동 매칭과 컬러맵 시각화가 실제로 구동되어 가치를 전달하는 웹사이트 형태의 MVP를 빠르게 완성하는 데 집중한다.

#### 다) 빠른 피드백과 협력

대학생 사용자를 대상으로 하는 서비스인 만큼, 초기 기획된 조건에만 매몰되지 않고 초기 배포된 웹사이트를 통해 시간표 수기 입력 패턴이나 뷰어 직관성에 대한 피드백을 지속적으로 교환하며 실제 요구사항을 서비스에 반영한다. 또한 초기 설계가 일부 변경되더라도 전체 구조에 영향

을 주지 않도록 유연한 설계를 유지한다.

**라) 변화에 대응**

고정된 계획을 고수하기보다 프로젝트 진행 중 불가피한 변화를 긍정적으로 수용한다. 실제로 초기 설계였던 DB 미구축 구조에서 확장성을 위한 DB 도입으로 인프라가 변경된 것과 같이, 술적 궤도 수정이나 예외 처리 요구를 민첩하게 설계에 반영한다.

# 4. 설계 평가

본 프로젝트의 설계 평가는 제안된 시스템이 초기 설정한 정량적 목표와 현실적 제한조건을 성공적으로 달성할 수 있는지 검증하기 위해 다음과 같은 기준으로 진행된다.

## 가. 핵심 알고리즘 성능 및 정확도 평가

1) 평가 내용: 참가자 수와 시간 슬롯 수가 증가할 때 기존 슬롯별 순회 방식과 비트마스킹 방식의 실행 시간을 비교하여, 워드 단위 연산이 실제 연산량을 얼마나 절감하는지 평가한다.
2) 검증 방법: 참여 인원(N)을 극단적으로 늘린 데이터를 주입하여 연산 속도를 측정하고, 선형 시간 복잡도 O(N)을 유지하며 수 초 이내에 응답하는지 확인한다. 또한, '이동 여유 시간 '과 '최소 회의 시간' 조건이 반영된 최종 교집합 결과가 기준 알고리즘과 결과가 일치하는지를 검증한다.

## 나. 예외 상황 처리 및 차선책 도출 검증

1) 평가 내용: 전원의 일정이 겹치지 않는 상태나 사용자의 비정상적인 입력 등 다양한 예외 상황에서의 시스템 안정성을 평가한다.
2) 검증 방법: 공통 공강 시간이 '0'인 상황을 시뮬레이션하여, 시스템이 멈추지 않고 전체 인원 중 최대 다수가 참석 가능한 차선책을 정상적으로 제안하는지 테스트한다. 또한 시간표 미입력자가 전원 참석 가능으로 연산되지 않도록 로직이 정상 작동하는지 확인한다.

## 다. 데이터 무결성 및 변환 평가

1) 평가 내용: 외부 시간표 데이터(이미지 OCR 결과 또는 수기 입력)가 서버 내에서 규격화된 포맷으로 손실 없이 변환되는지 평가한다.
2) 검증 방법: 입력된 주간 시간표 데이터가 참석 가능(1)과 불가능(0) 상태의 120비트 비트 벡터로 정확히 변환되는지 확인한다. 또한 원본 일정 데이터가 데이터베이스에 정상적으로 저장·조회되고, 매칭 과정에서 비트 벡터로 변환하여 공통 참석 가능 시간을 정확히 계산하는지 테스트를 통해 검증한다.

## 라. 활용성 평가

1) 평가 내용: 실제 사용자가 서비스를 이용할 때 느끼는 편의성과 일정 조율의 실질적인 소요 시간 단축 여부를 평가한다.
2) 검증 방법: 겹치는 인원 누적 수에 비례한 컬러맵 시각화가 사용자에게 정확히 보이는지 확인한다. 또한, 소셜 로그인부터 모임 방 생성, 시간표 입력, 알고리즘 매칭, 투표, 최종 일정 확정에 이르는 통합 테스트를 수행하여 끊김이 없는지 확인한다.

# 5. 설계/프로젝트 계획

## 가. 서비스 개요

'틈(Teum)'은 에브리타임 시간표 이미지 등을 활용해 수동 입력의 번거로움을 없애고, 대학생 팀 프로젝트 및 스터디를 위한 최적의 공강 시간을 자동으로 추천해 주는 일정 조율 웹 서비스이다.

## 나. 앞으로의 개발 계획

| 작업 항목 | 세부 설명 |
|---|---|
| 상세 설계 보완 및 개발 환경 구축 | -초대 대상과 수락 상태를 관리하는 구조를 보완하고, 데이터베이스와 API 명세를 확정한다.<br>-팀원별 역할과 개발 환경을 정비한다. |
| 사용자 및 모임 관리 구현 | -소셜 로그인, 모임 생성, 팀원 초대 및 수락 기능을 구현하고, 구성원의 참여 상태를 관리한다. |
| 시간표 입력 및 데이터 변환 구현 | -시간표 이미지 인식과 수기 입력 기능을 구현한다.<br>-인식 결과를 사용자가 확인·수정하도록 하고, 입력된 일정을 규격화된 시간 슬롯과 비트 벡터로 변환한다. |
| 공강 추천 알고리즘 구현 | -비트 AND 연산으로 공통 공강을 계산하고, 이동 여유 시간과 최소 회의 시간을 반영하여 상위 3개의 후보를 추천한다.<br>-공통 시간이 없으면 최대 인원이 참석할 수 있는 차선안을 제시한다. |
| 투표 및 일정 확정 구현 | -추천 후보에 대한 투표와 결과 집계 기능을 구현한다.<br>-일정 확정 전 최신 시간표와의 충돌을 재검사하고, 시간표나 구성원 변경 시 재계산할 수 있도록 한다. |
| 화면 연동 및 시각화 | -참석 가능 인원을 컬러맵으로 표현하고, 시간표 입력부터 일정 확정까지의 화면과 서버 기능을 연동한다. |
| 검증 및 최종 시연 준비 | -추천 결과의 정확도, 응답 시간, 데이터 변환 및 예외 처리를 검증한다.<br>-사용자 피드백을 반영하여 오류와 사용성을 개선하고 최종 보고서와 시연 자료를 작성한다. |

<표35 - 앞으로의 개발 계획>

## 다. 예상 일정

| 주차 | 작업 내용 |
|---|---|
| 8주차 | ERD 및 클래스 다이어그램 보완, API 명세 확정, 개발 환경 및 데이터베이스 구축. 소셜 로그인, 모임 생성, 팀원 초대·수락 및 시간표 수기 입력 기능 구현 |
| 9주차 | 시간표 이미지 인식 및 결과 수정 기능, 비트 벡터 변환 구현. 이동 여유 시간과 최소 회의 시간을 반영한 공통 공강 계산, 상위 후보 및 차선안 추천 기능 구현 |
| 10주차 | 투표·집계, 일정 확정 전 재검사 및 재계산 기능 구현. 컬러맵 시각화와 전체 화면·서버 연동 후 주요 이용 흐름 통합 테스트 |
| 11주차 | 알고리즘 정확도·성능 및 예외 처리 검증, 오류 수정과 사용자 피드백 반영. 최종 동작 점검 및 배포, 최종 보고서 작성과 시연 준비 |

<표36 - 예상 일정>

## 6. 참고자료

가. 학술적 근거 및 참고 문헌

1) 참고 문헌 1: Van Kessel, Philippe, and Claude-Guy Quimper. Filtering Algorithms Based on the Word-RAM Model. Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence, 2012, pp. 577-583.

가) [적용 부분]: 워드 단위 연산을 활용하는 Word-RAM 모델과 비트셋 기반 연산의 시간 복잡도를 설명하는 근거로 활용하였다. 특히 비트 연산을 통해 하나의 워드에 포함된 여러 값을 동시에 처리할 수 있다는 원리를 주간 시간표의 비트 벡터 표현과 공통 참석 가능 시간 계산에 적용하였다.

2) 참고 문헌 2: OpenDSA Project Contributors. Bit Vectors for Representing Sets. OpenDSA Data Structures and Algorithms Modules Collection, Section 22.5.

가) [적용 부분]: 비트 벡터로 집합을 표현하고 비트 AND 연산으로 집합의 교집합을 계산하는 원리를 참고하였다. 이를 각 참여자의 참석 가능 시간 집합을 비트 벡터로 표현하고, 전체 참여자의 공통 참석 가능 시간을 계산하는 방식에 적용하였다.

나. 애자일(Agile) MVP (최소 기능 제품) 개발 방법론

1) 애자일방법론을 적용한 정보시스템의 감리품질향상에 관한 연구- 박동아, 박만곤

가) [적용 부분]: 3page 2-3. 애자일 방법론의 특징 및 종류 일부 발췌 후 활용 하였음.